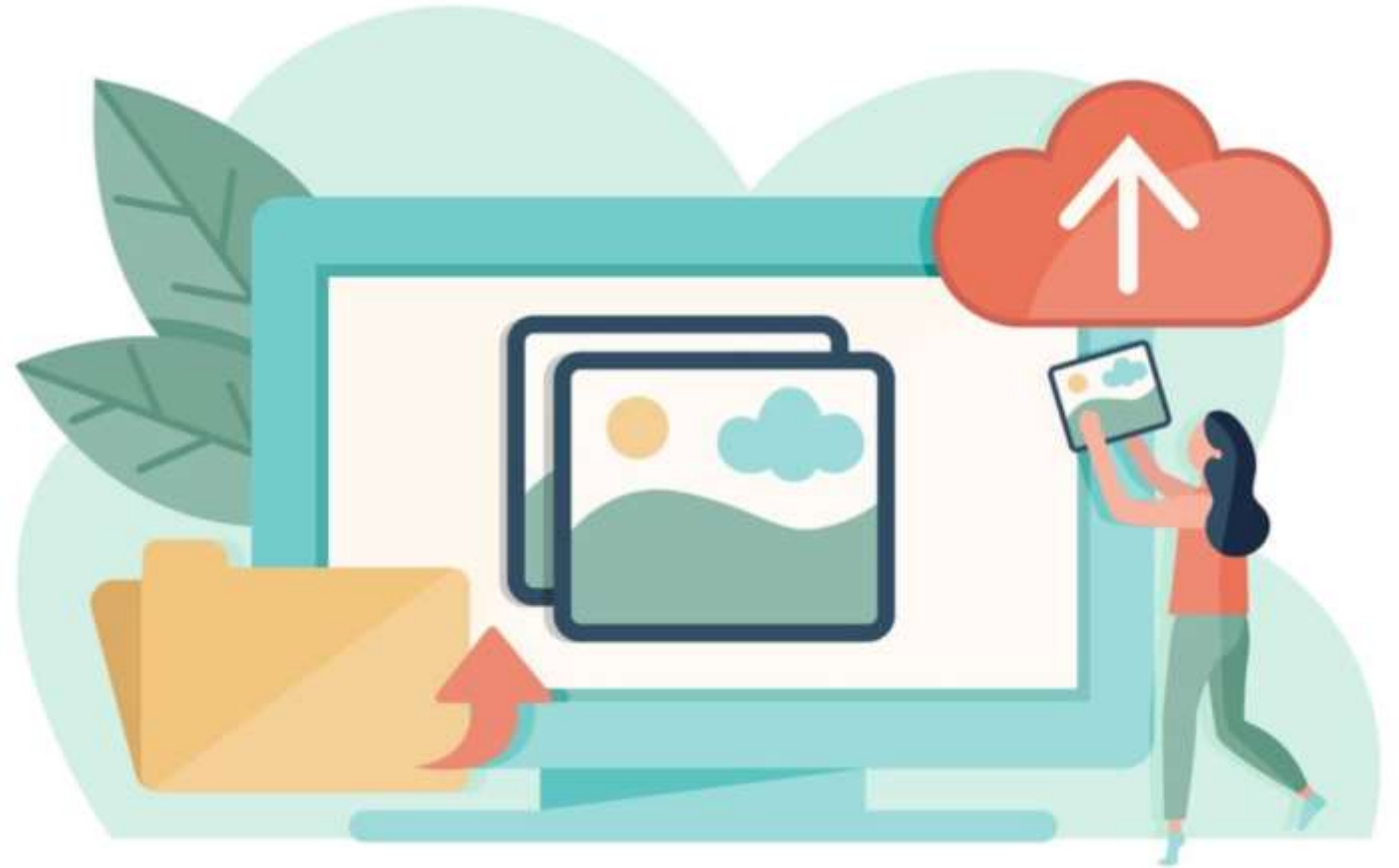


File Handling in Python



Introduction to File Handling

File handling is the process of performing operations like creating, opening, reading, writing, and closing a file using Python.

◆ Importance:

- 📁 Allows interaction with external files
- 💾 Used for persistent storage of data
- 📖 Read from or write to text, CSV, logs, or config files

□ Real-life Use Cases:

- Logging user activity
- Storing application configuration
- Reading large datasets



File Handling Functions

Python provides built-in functions for file handling:

Function	Purpose
open()	Opens a file
read()	Reads content
write()	Writes to a file
close()	Closes the file



File Modes in Python

When opening a file, we specify a **mode**:

Mode	Description
'r'	Read (default)
'w'	Write (overwrites if exists)
'a'	Append
'x'	Create (error if exists)
'b'	Binary mode
't'	Text mode (default)

Example:

```
f = open("data.txt", "r")
```



Reading from a File

Three ways to read:

```
f = open("demo.txt", "r")  
print(f.read())    # Reads entire file  
print(f.readline()) # Reads one line  
print(f.readlines()) # Reads lines into a list  
f.close()
```

✦ Don't forget to close the file!



Writing to a File

```
f = open("output.txt", "w")  
f.write("Python File Handling is powerful!")  
f.close()
```

 If file exists, it will be **overwritten**.

Creates the file if it doesn't exist.



Appending to a File

```
f = open("output.txt", "a")  
f.write("\nAppending a new line.")  
f.close()
```

💡 Keeps existing content intact and adds new text.



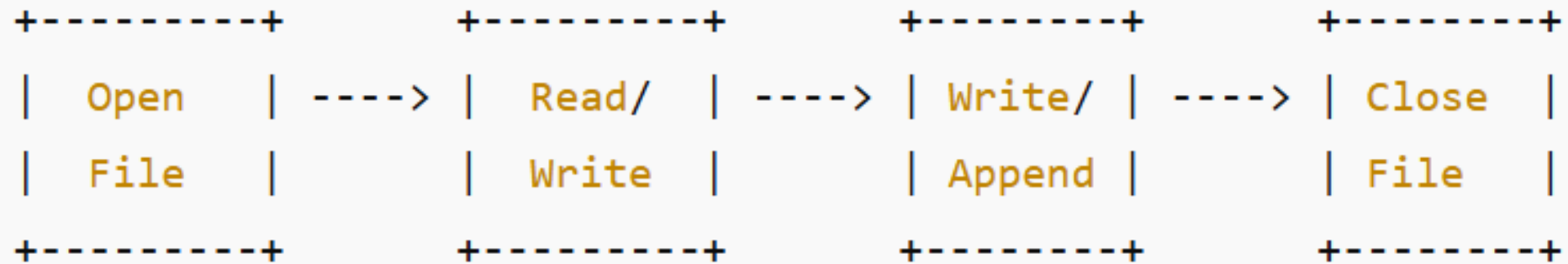
Using with Statement

```
with open("example.txt", "r") as f:  
    content = f.read()  
    print(content)
```

- 🔒 Auto-closes file even if errors occur
- ✂ Preferred way of file handling



➤ File Handling Workflow (Flowchart)



Combined Read-Write Example

```
# Write to file

with open("demo.txt", "w") as f:
    f.write("Welcome to Python file handling.")

# Read the file

with open("demo.txt", "r") as f:
    print(f.read())
```

☐ **Output:**

```
Welcome to Python file handling.
```



Real-world Problem – Student Record

Problem: Write a program to add student names and then read them.

```
# Adding student names

with open("students.txt", "a") as f:
    f.write("Ritesh\n")
    f.write("Aarav\n")

# Reading student names

with open("students.txt", "r") as f:
    for line in f:
        print("Student:", line.strip())
```



File Checking with os Module

Check if file exists before performing operations:

```
import os

if os.path.exists("demo.txt"):
    print("Yes, file exists!")
else:
    print("No, file not found.")
```





Deleting Files

Delete a file permanently:

```
import os  
os.remove("demo.txt")
```

⚠ This cannot be undone. Use carefully.



➤ Handling File Errors (Try-Except)

```
try:  
    with open("no_file.txt", "r") as f:  
        print(f.read())  
except FileNotFoundError:  
    print("File not found!")
```

✓ Prevents the program from crashing if the file doesn't exist.



Practice Problem – Write & Read Marks

Problem: Write student marks to a file, then read and calculate average.

```
with open("marks.txt", "w") as f:  
    f.write("85\n90\n78\n")  
  
with open("marks.txt", "r") as f:  
    marks = f.readlines()  
    marks = [int(m) for m in marks]  
    print("Average =", sum(marks)/len(marks))
```



